RefactoringAgileArchitectureAboutThoughtworks

Table of Contents
Inversion of Control Containers and the Dependency Injection pattern
In the Java community there's been a rush of lightweight containers that help to assemble components fro

23 January 2004

Photo of Martin Fowler
Martin Fowler
design

object collaboration design

application architecture

Translations: Chinese
Contents
Components and Services
A Naive Example
Inversion of Control
Forms of Dependency Injection
Constructor Injection with PicoContainer
Setter Injection with Spring
Interface Injection
Using a Service Locator
Using a Segregated Interface for the Locator
A Dynamic Service Locator
Using both a locator and injection with Avalon
Deciding which option to use
Service Locator vs Dependency Injection
Constructor versus Setter Injection
Code or configuration files
Separating Configuration from Use
Some further issues
Concluding Thoughts
One of the entertaining things about the enterprise Java world is the huge amount of activity in buildin

Underlying these containers are a number of interesting design principles, things that go beyond both th

Components and Services
The topic of wiring elements together drags me almost immediately into the knotty terminology problems t

I use component to mean a glob of software that's intended to be used, without change, by an application

A service is similar to a component in that it's used by foreign applications. The main difference is th

I mostly use service in this article, but much of the same logic can be applied to local components too.

A Naive Example
To help make all of this more concrete I'll use a running example to talk about all of this. Like all of

In this example I'm writing a component that provides a list of movies directed by a particular director

class MovieLister...

```
  public Movie[] moviesDirectedBy(String arg) {
      List allMovies = finder.findAll();
      for (Iterator it = allMovies.iterator(); it.hasNext();) {
          Movie movie = (Movie) it.next();
          if (!movie.getDirector().equals(arg)) it.remove();
      }
      return (Movie[]) allMovies.toArray(new Movie[allMovies.size()]);
  }
```

The implementation of this function is naive in the extreme, it asks a finder object (which we'll get to

The real point of this article is this finder object, or particularly how we connect the lister object w

```
public interface MovieFinder {
    List findAll();
}
```

Now all of this is very well decoupled, but at some point I have to come up with a concrete class to act

class MovieLister...

```
  private MovieFinder finder;
  public MovieLister() {
    finder = new ColonDelimitedMovieFinder("movies1.txt");
  }
```

The name of the implementation class comes from the fact that I'm getting my list from a colon delimited
Now if I'm using this class for just myself, this is all fine and dandy. But what happens when my friend

Figure 1: The dependencies using a simple creation in the lister class

Figure 1 shows the dependencies for this situation. The MovieLister class is dependent on both the Movie

In my book P of EAA, we described this situation as a Plugin. The implementation class for the finder is

Expanding this into a real system, we might have dozens of such services and components. In each case we

So the core problem is how do we assemble these plugins into an application? This is one of the main pro

Inversion of Control

When these containers talk about how they are so useful because they implement “Inversion of Control” I

The question is: “what aspect of control are they inverting?” When I first ran into inversion of control

For this new breed of containers the inversion is about how they lookup a plugin implementation. In my n

As a result I think we need a more specific name for this pattern. Inversion of Control is too generic a

I'm going to start by talking about the various forms of dependency injection, but I'll point out now th

Forms of Dependency Injection

The basic idea of the Dependency Injection is to have a separate object, an assembler, that populates a

Figure 2: The dependencies for a Dependency Injector

There are three main styles of dependency injection. The names I'm using for them are Constructor Inject

Constructor Injection with PicoContainer

I'll start with showing how this injection is done using a lightweight container called PicoContainer. I

PicoContainer uses a constructor to decide how to inject a finder implementation into the lister class.

```
class MovieLister...

  public MovieLister(MovieFinder finder) {
      this.finder = finder;
  }
```

The finder itself will also be managed by the pico container, and as such will have the filename of the

```
class ColonMovieFinder...

  public ColonMovieFinder(String filename) {
      this.filename = filename;
  }
```

The pico container then needs to be told which implementation class to associate with each interface, an

```
private MutablePicoContainer configureContainer() {
    MutablePicoContainer pico = new DefaultPicoContainer();
    Parameter[] finderParams =  {new ConstantParameter("movies1.txt")};
    pico.registerComponentImplementation(MovieFinder.class, ColonMovieFinder.class, finderParams);
    pico.registerComponentImplementation(MovieLister.class);
    return pico;
}
```

This configuration code is typically set up in a different class. For our example, each friend who uses

To use the container you write code something like this.

```
public void testWithPico() {
    MutablePicoContainer pico = configureContainer();
    MovieLister lister = (MovieLister) pico.getComponentInstance(MovieLister.class);
    Movie[] movies = lister.moviesDirectedBy("Sergio Leone");
    assertEquals("Once Upon a Time in the West", movies[0].getTitle());
}
```

Although in this example I've used constructor injection, PicoContainer also supports setter injection,

Setter Injection with Spring

The Spring framework is a wide ranging framework for enterprise Java development. It includes abstractio

To get my movie lister to accept the injection I define a setting method for that service

```
class MovieLister...

  private MovieFinder finder;
public void setFinder(MovieFinder finder) {
  this.finder = finder;
}
```

Similarly I define a setter for the filename.

```
class ColonMovieFinder...
```

```
  public void setFilename(String filename) {
      this.filename = filename;
  }
```
The third step is to set up the configuration for the files. Spring supports configuration through XML f

```
<beans>
    <bean id="MovieLister" class="spring.MovieLister">
        <property name="finder">
            <ref local="MovieFinder"/>
        </property>
    </bean>
    <bean id="MovieFinder" class="spring.ColonMovieFinder">
        <property name="filename">
            <value>movies1.txt</value>
        </property>
    </bean>
</beans>
```
The test then looks like this.

```
public void testWithSpring() throws Exception {
    ApplicationContext ctx = new FileSystemXmlApplicationContext("spring.xml");
    MovieLister lister = (MovieLister) ctx.getBean("MovieLister");
    Movie[] movies = lister.moviesDirectedBy("Sergio Leone");
    assertEquals("Once Upon a Time in the West", movies[0].getTitle());
}
```
Interface Injection

The third injection technique is to define and use interfaces for the injection. Avalon is an example of

With this technique I begin by defining an interface that I'll use to perform the injection through. Her

```
public interface InjectFinder {
    void injectFinder(MovieFinder finder);
}
```
This interface would be defined by whoever provides the MovieFinder interface. It needs to be implemente

```
class MovieLister implements InjectFinder

  public void injectFinder(MovieFinder finder) {
      this.finder = finder;
  }
```
I use a similar approach to inject the filename into the finder implementation.

```
public interface InjectFinderFilename {
    void injectFilename (String filename);
}
class ColonMovieFinder implements MovieFinder, InjectFinderFilename...

  public void injectFilename(String filename) {
      this.filename = filename;
  }
```
Then, as usual, I need some configuration code to wire up the implementations. For simplicity's sake I'l

```
class Tester...

  private Container container;

   private void configureContainer() {
     container = new Container();
     registerComponents();
     registerInjectors();
     container.start();
  }
```
This configuration has two stages, registering components through lookup keys is pretty similar to the c

```
class Tester...

  private void registerComponents() {
    container.registerComponent("MovieLister", MovieLister.class);
    container.registerComponent("MovieFinder", ColonMovieFinder.class);
  }
```
A new step is to register the injectors that will inject the dependent components. Each injection interf

```
class Tester...

  private void registerInjectors() {
    container.registerInjector(InjectFinder.class, container.lookup("MovieFinder"));
    container.registerInjector(InjectFinderFilename.class, new FinderFilenameInjector());
  }
public interface Injector {
  public void inject(Object target);
}
```

When the dependent is a class written for this container, it makes sense for the component to implement

```
class ColonMovieFinder implements Injector...

  public void inject(Object target) {
    ((InjectFinder) target).injectFinder(this);
  }
class Tester...

  public static class FinderFilenameInjector implements Injector {
    public void inject(Object target) {
      ((InjectFinderFilename)target).injectFilename("movies1.txt");
    }
    }
```

The tests then use the container.

```
class Tester…

  public void testIface() {
    configureContainer();
    MovieLister lister = (MovieLister)container.lookup("MovieLister");
    Movie[] movies = lister.moviesDirectedBy("Sergio Leone");
    assertEquals("Once Upon a Time in the West", movies[0].getTitle());
  }
```

The container uses the declared injection interfaces to figure out the dependencies and the injectors to

Using a Service Locator

The key benefit of a Dependency Injector is that it removes the dependency that the MovieLister class ha

The basic idea behind a service locator is to have an object that knows how to get hold of all of the se

Figure 3: The dependencies for a Service Locator

In this case I'll use the ServiceLocator as a singleton Registry. The lister can then use that to get th

```
class MovieLister...

  MovieFinder finder = ServiceLocator.movieFinder();
class ServiceLocator...

  public static MovieFinder movieFinder() {
      return soleInstance.movieFinder;
  }
  private static ServiceLocator soleInstance;
  private MovieFinder movieFinder;
```

As with the injection approach, we have to configure the service locator. Here I'm doing it in code, but

```
class Tester...

  private void configure() {
      ServiceLocator.load(new ServiceLocator(new ColonMovieFinder("movies1.txt")));
  }
class ServiceLocator...

  public static void load(ServiceLocator arg) {
      soleInstance = arg;
  }

  public ServiceLocator(MovieFinder movieFinder) {
      this.movieFinder = movieFinder;
  }
```

Here's the test code.

```
class Tester...

  public void testSimple() {
      configure();
      MovieLister lister = new MovieLister();
      Movie[] movies = lister.moviesDirectedBy("Sergio Leone");
      assertEquals("Once Upon a Time in the West", movies[0].getTitle());
  }
```

I've often heard the complaint that these kinds of service locators are a bad thing because they aren't

For a more sophisticated locator I can subclass service locator and pass that subclass into the registry

A way to think of this is that service locator is a registry not a singleton. A singleton provides a sim

Using a Segregated Interface for the Locator

One of the issues with the simple approach above, is that the MovieLister is dependent on the full servi
In this situation the provider of the lister would also provide a locator interface which it needs to ge

```
public interface MovieFinderLocator {
    public MovieFinder movieFinder();
```
The locator then needs to implement this interface to provide access to a finder.

```
MovieFinderLocator locator = ServiceLocator.locator();
MovieFinder finder = locator.movieFinder();
public static ServiceLocator locator() {
     return soleInstance;
 }
 public MovieFinder movieFinder() {
     return movieFinder;
 }
 private static ServiceLocator soleInstance;
 private MovieFinder movieFinder;
```
You'll notice that since we want to use an interface, we can't just access the services through static m

A Dynamic Service Locator

The above example was static, in that the service locator class has methods for each of the services tha

In this case, the service locator uses a map instead of fields for each of the services, and provides ge

```
class ServiceLocator...

  private static ServiceLocator soleInstance;
  public static void load(ServiceLocator arg) {
      soleInstance = arg;
  }
  private Map services = new HashMap();
  public static Object getService(String key){
      return soleInstance.services.get(key);
  }
  public void loadService (String key, Object service) {
      services.put(key, service);
  }
```
Configuring involves loading a service with an appropriate key.

```
class Tester...

  private void configure() {
      ServiceLocator locator = new ServiceLocator();
      locator.loadService("MovieFinder", new ColonMovieFinder("movies1.txt"));
      ServiceLocator.load(locator);
  }
```
I use the service by using the same key string.

```
class MovieLister...

  MovieFinder finder = (MovieFinder) ServiceLocator.getService("MovieFinder");
```
On the whole I dislike this approach. Although it's certainly flexible, it's not very explicit. The only

Using both a locator and injection with Avalon

Dependency injection and a service locator aren't necessarily mutually exclusive concepts. A good exampl

Berin Loritsch sent me this simple version of my running example using Avalon.

```
public class MyMovieLister implements MovieLister, Serviceable {
    private MovieFinder finder;

    public void service( ServiceManager manager ) throws ServiceException {
        finder = (MovieFinder)manager.lookup(“finder”);
    }
```

The service method is an example of interface injection, allowing the container to inject a service mana

Deciding which option to use

So far I've concentrated on explaining how I see these patterns and their variations. Now I can start ta

Service Locator vs Dependency Injection

The fundamental choice is between Service Locator and Dependency Injection. The first point is that both

Inversion of control is a common feature of frameworks, but it's something that comes at a price. It ten

The key difference is that with a Service Locator every user of a service has a dependency to the locato

Using dependency injection can help make it easier to see what the component dependencies are. With depe

A lot of this depends on the nature of the user of the service. If you are building an application with

The difference comes if the lister is a component that I'm providing to an application that other people

Since with an injector you don't have a dependency from a component to the injector, the component canno

A common reason people give for preferring dependency injection is that it makes testing easier. The poi

Of course the testing problem is exacerbated by component environments that are very intrusive, such as

So the primary issue is for people who are writing code that expects to be used in applications outside

Constructor versus Setter Injection

For service combination, you always have to have some convention in order to wire things together. The a

Interface injection is more invasive since you have to write a lot of interfaces to get things all sorte

The choice between setter and constructor injection is interesting as it mirrors a more general issue wi

My long running default with objects is as much as possible, to create valid objects at construction tim

Another advantage with constructor initialization is that it allows you to clearly hide any fields that

But with any situation there are exceptions. If you have a lot of constructor parameters things can look

If you have multiple ways to construct a valid object, it can be hard to show this through constructors,

Constructors also suffer if you have simple parameters such as strings. With setter injection you can gi

If you have multiple constructors and inheritance, then things can get particularly awkward. In order to

Despite the disadvantages my preference is to start with constructor injection, but be ready to switch t

This issue has led to a lot of debate between the various teams who provide dependency injectors as part

Code or configuration files

A separate but often conflated issue is whether to use configuration files or code on an API to wire up

A contrasting case is where the assembly is quite complex, involving conditional steps. Once you start g

I often think that people are over-eager to define configuration files. Often a programming language mak

It's often said that configuration files shouldn't use a programing language because they need to be edi

One thing we're seeing in the Java world at the moment is a cacophony of configuration files, where ever

My advice here is to always provide a way to do all configuration easily with a programmatic interface,

Separating Configuration from Use

The important issue in all of this is to ensure that the configuration of services is separated from the

If this separation is useful within a single code base, it's especially vital when you're using foreign

Some further issues

In this article, I've concentrated on the basic issues of service configuration using Dependency Injecti

You can find out a lot more about these ideas by looking at the web sites devoted to the lightweight con

Concluding Thoughts

The current rush of lightweight containers all have a common underlying pattern to how they do service a

If you use Dependency Injection there are a number of styles to choose between. I would suggest you foll

The choice between Service Locator and Dependency Injection is less important than the principle of sepa

Acknowledgments

My sincere thanks to the many people who've helped me with this article. Rod Johnson, Paul Hammant, Joe

Significant Revisions

© Martin Fowler | Disclosures